LAPORAN TUGAS PELATIHAN LAB B401

Robot Humanoid Obstacle Avoidance dengan Lidar 360° di Atas Kepala

Gait engine per sendi, navigasi koridor Lidar, dan eksplorasi berbasis kebaruan di Webots



Gambar 1. Robot humanoid NAO (25 DOF) dengan Lidar 360° di atas kepala, di arena 5 × 5 m berisi 12 obstacle.

Disusun oleh: **Ahnaf**

Repositori: https://github.com/ahnafyura/webots-obstacle-avoidance-b401

Webots R2025a · Python 3 · ROS 2 Jazzy (RViz2)

November 2026

1. Pendahuluan

1.1 Latar Belakang

Pada modul pelatihan, robot dirakit sebagai robot beroda *line tracer*. Tugas ini meminta robot **obstacle avoidance** dengan bentuk dan sensor bebas, di arena yang berisi obstacle di bagian tengah, bukan hanya dinding pinggir. Proyek ini memakai robot **humanoid** (berjalan dengan dua kaki). Humanoid jauh lebih sulit dibanding robot beroda: robot harus tetap seimbang di setiap langkah, setiap sendi harus digerakkan dengan benar, dan robot tidak dapat berbelok atau berhenti seketika. Sensor utamanya Lidar 360° yang dipasang di atas kepala.

1.2 Ketentuan Tugas dan Pemenuhannya

Tabel 1. Pemenuhan ketentuan tugas

Ketentuan	Implementasi	Status
Robot obstacle avoidance	Humanoid berjalan otonom dan menghindari obstacle. Diuji 32 kali × 10 menit (5.3 jam simulasi) dengan hasil 0 tabrakan dan 0 kali jatuh.	✔
Bentuk robot bebas	Humanoid NAO 25 DOF. Setiap sendi dikendalikan oleh gait engine buatan sendiri (transisi halus, siklus jalan kontinu, belok sambil jalan).	✔
Obstacle di arena, tidak hanya pinggiran	12 obstacle setinggi 0.8 m di dalam arena 5 × 5 m: kotak, kotak miring, tabung, dinding, dan siku-L (sudut cekung).	✔
Sensor bebas	Lidar 360° di kepala (utama), 2 sonar, IMU, gyro, accelerometer, GPS, sensor gaya kaki (FSR), kamera, dan sensor posisi sendi. Semua dapat ditampilkan di RViz2.	✔

1.3 Tujuan

- Membuat humanoid yang dapat berjalan lurus, berbelok sambil berjalan, berputar di tempat, dan mundur tanpa jatuh.
- Menghindari obstacle memakai Lidar 360° di atas kepala, dengan sonar sebagai konfirmasi.
- Meningkatkan cakupan (coverage) eksplorasi arena dan membuktikannya secara kuantitatif dibandingkan eksplorasi acak.
- Menampilkan seluruh data sensor dan model robot di RViz2.

2. Dasar Teori

2.1 Robot Humanoid dan Gait

Gait adalah pola gerak sendi yang berulang saat berjalan. Satu siklus berisi dua langkah. Setiap langkah terdiri dari fase *double support* (kedua kaki menapak) dan *single support* (satu kaki terangkat). Saat satu kaki terangkat, pusat massa harus digeser ke atas kaki tumpu agar robot tidak jatuh. Itulah fungsi ayunan sendi *HipRoll*/*AnkleRoll* ke kiri dan ke kanan. Gait pada proyek ini berbasis *keyframe*: sudut setiap sendi disimpan setiap 40 ms, lalu diinterpolasi linear di antara keyframe. Untuk waktu t di antara keyframe k (waktu t_k, sudut q_k) dan k+1:

$$q(t) = q_k + (q_{k+1} - q_k) \cdot (t - t_k) / (t_{k+1} - t_k)$$

2.2 Lidar

Lidar mengukur jarak dari waktu tempuh pantulan laser: $d = c \cdot t/2$. Lidar 2D berputar menghasilkan deret jarak per sudut. Dari hasil uji di Webots, Lidar 360 sinar ini mengurutkan data sebagai berikut: indeks 0 mengarah ke belakang, lalu searah jarum jam (90 = kiri, 180 = depan, 270 = kanan). Jadi sudut sinar ke-i adalah $\theta_i = \pi - i \cdot (2\pi/360)$, dan titiknya dalam frame robot adalah $(x, y) = (r_i \cos \theta_i, r_i \sin \theta_i)$ dengan x ke depan dan y ke kiri.

2.3 Sensor Pendukung

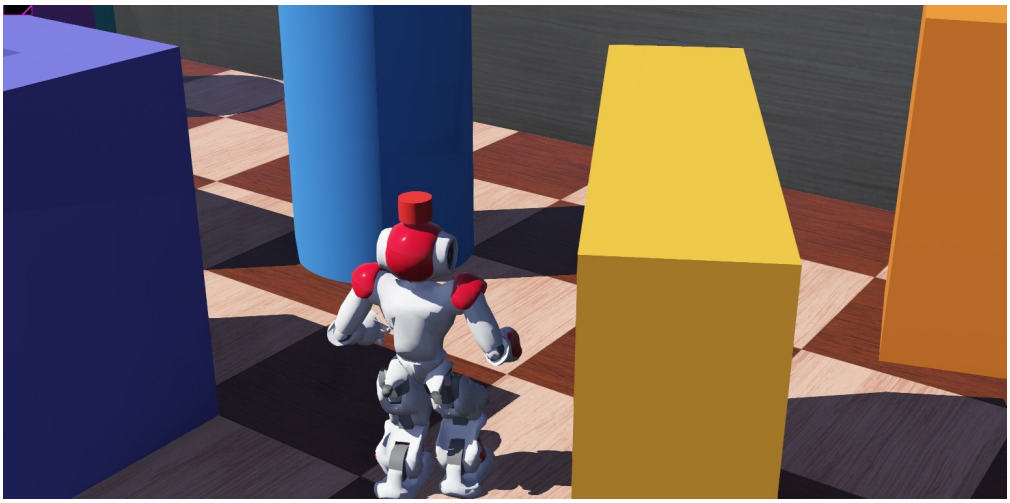
- **Sonar** (ultrasonik): mengukur jarak dari pantulan suara. Pancarannya lebar (sekitar 60°), sehingga baik untuk deteksi dekat tetapi kurang tepat arahnya.
- **IMU** (InertialUnit): memberi orientasi roll, pitch, dan yaw. Roll dan pitch dipakai untuk mendeteksi robot jatuh, yaw sebagai arah hadap.
- **Gyro** dan **accelerometer**: kecepatan sudut dan percepatan badan.
- **GPS**: posisi robot di arena, dipakai untuk memori eksplorasi (sel mana yang sudah dikunjungi).
- **FSR** (force sensitive resistor) di telapak kaki: gaya tapak, menunjukkan kaki mana yang menumpu.
- **Sensor posisi sendi**: sudut aktual setiap sendi, dipakai untuk transisi halus dan model robot di RViz2.

2.4 Navigasi Reaktif dan Eksplorasi

Navigasi reaktif memutuskan gerak dari pembacaan sensor saat itu. Agar robot tidak hanya menghindar tetapi juga menjelajahi arena, dipakai memori kunjungan berbentuk grid. Arah menuju sel yang jarang dikunjungi diberi nilai lebih tinggi, mirip prinsip eksplorasi *frontier*. Kinerja eksplorasi diukur dengan **coverage**, yaitu persentase sel arena yang dapat dilalui dan pernah dikunjungi.

3. Perancangan Robot

Robot yang dipakai adalah model humanoid NAO dari Webots: tinggi 57 cm, 25 derajat kebebasan (DOF). Lidar ditambahkan pada *headSlot*, yaitu di atas kepala, 0.57 m dari lantai. Dari posisi ini Lidar dapat melihat ke segala arah tanpa terhalang badan atau lengan robot. InertialUnit tambahan dipasang di *bodySlot*, karena InertialUnit bawaan NAO tidak mengeluarkan sudut yaw.



Gambar 2. Humanoid saat berjalan. Silinder merah di atas kepala adalah Lidar 360°.

Tabel 2. Sendi dan sensor robot humanoid

Bagian	Sendi / device	Keterangan
Kaki (12 sendi)	HipYawPitch, HipRoll, HipPitch, KneePitch, AnklePitch, AnkleRoll (kiri & kanan)	Digerakkan gait engine setiap 20 ms
Lengan (8 sendi)	ShoulderPitch, ShoulderRoll, ElbowYaw, ElbowRoll (kiri & kanan)	Ditahan di samping badan saat berjalan
Kepala (2 sendi)	HeadYaw, HeadPitch	Ditahan 0 agar Lidar tetap datar
Lidar 360°	Lidar `lidar`	360 sinar, 0.06–4 m, noise 0.2%, tinggi 0.57 m
Sonar ×2	`Sonar/Left`, `Sonar/Right`	Jangkauan 0.25–2.55 m, di dada
IMU	InertialUnit `imu_body`	Roll, pitch, yaw
Gyro, accelerometer	`gyro`, `accelerometer`	Dipublish ke ROS 2
GPS	`gps`	Posisi untuk memori eksplorasi
FSR kaki ×2	`LFsr`, `RFsr`	Gaya tapak 3 sumbu
Kamera	`CameraTop`	320 × 240, dipublish ke ROS 2

4. Gait Engine: Membuat Humanoid Berjalan dengan Baik

4.1 Masalah pada Motion Bawaan

Webots menyediakan file gerak (.motion) untuk NAO. Saat diuji langsung, gerakan *Forwards50* (jalan panjang) membuat robot **jatuh ke depan pada detik ke-1.2** (pitch badan 68°). Penyebabnya, file motion langsung melompat ke pose pertamanya (lutut ditekuk 1.22 rad), sedangkan robot berdiri dengan kaki lurus. Lompatan sudut sebesar ini menyentak badan. Selain itu, file motion tidak dapat disambung menjadi jalan kontinu dan tidak ada gerak belok sambil berjalan. Karena itu dibuat gait engine sendiri (`gait.py`) yang mengendalikan setiap sendi secara langsung.

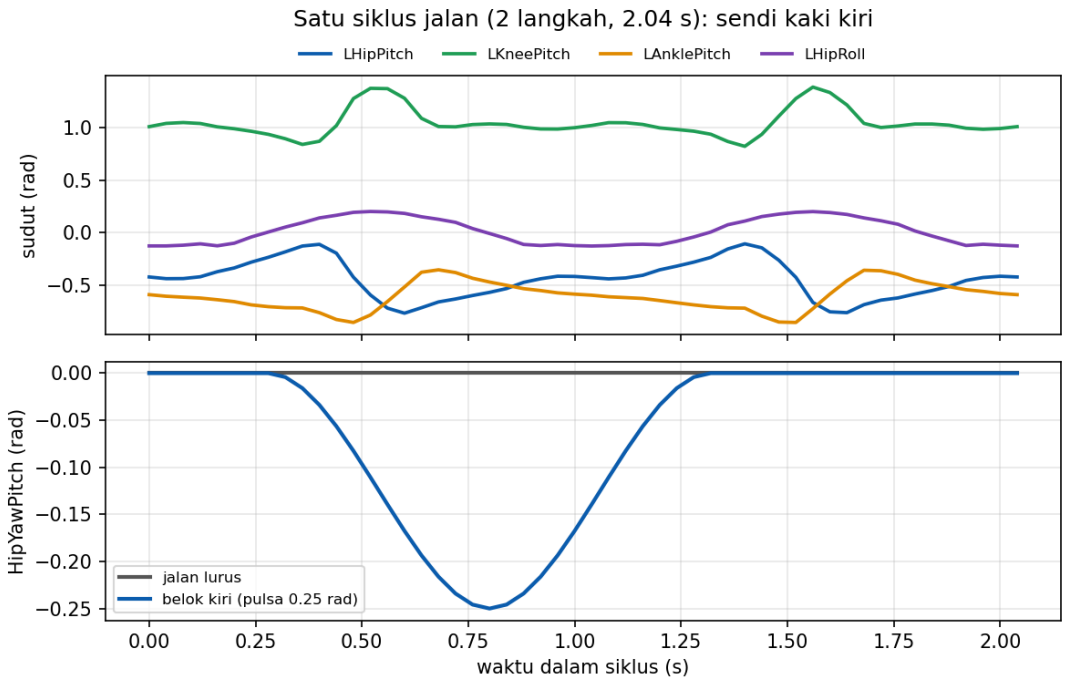
4.2 Transisi Halus Antar Gerakan

Sebelum klip gerak dimulai, setiap sendi j berpindah dari sudut target terakhir $q_{j,awal}$ ke sudut pertama klip $q_{j,1}$ selama T_b (0.4 s), memakai fungsi *smoothstep* yang kecepatan awal dan akhirnya nol, sehingga tidak ada sentakan:

$$u = t / T_b,\ s(u) = 3u^2 - 2u^3,\ q_j(t) = q_{j,awal} + (q_{j,1} - q_{j,awal}) \cdot s(u),\ 0 \le t \le T_b$$

4.3 Siklus Jalan Kontinu

Gerakan Forwards50 dianalisis dengan autokorelasi sudut *HipRoll*. Periode ayunannya 25.5 frame (1.02 s per langkah). Frame 51 dan frame 102 ternyata **identik** (selisih maksimum 0 rad untuk semua sendi). Jadi frame 51–102 adalah satu siklus dua langkah (2.04 s) yang dapat diulang tanpa sambungan kasar. Forwards50 dipotong menjadi tiga klip: *mulai* (frame 0–51), *siklus* (51–102, diulang selama jalan aman), dan *berhenti* (102–170).



Gambar 3. Atas: sudut sendi kaki kiri dalam satu siklus jalan. Bawah: sendi HipYawPitch, datar saat jalan lurus dan berpulsa saat belok kiri.

4.4 Belok Sambil Berjalan

Pada gerakan belok bawaan (TurnLeft40), sendi *HipYawPitch* dibuka sampai −0.48 rad saat satu kaki terangkat, lalu ditutup kembali saat kaki itu menapak. Pola yang sama disisipkan ke siklus jalan sebagai satu pulsa per siklus. Untuk belok kiri, pulsa naik pada frame 7–20 dan turun pada frame 20–33. Untuk belok kanan, pulsa naik pada frame 20–33 dan turun pada frame 33–46. Dengan amplitudo A = 0.25 rad dan k = indeks frame dalam siklus, belok kiri dirumuskan:

$$q_{\text{HipYawPitch}}(k) = q_{\text{siklus}}(k) - A \cdot s(p(k)),\ p(k) = (k-7)/13 \text{ untuk } 7 \le k < 20,\ p(k) = 1 - (k-20)/13 \text{ untuk } 20 \le k < 33,\ p(k) = 0 \text{ di luar itu}$$

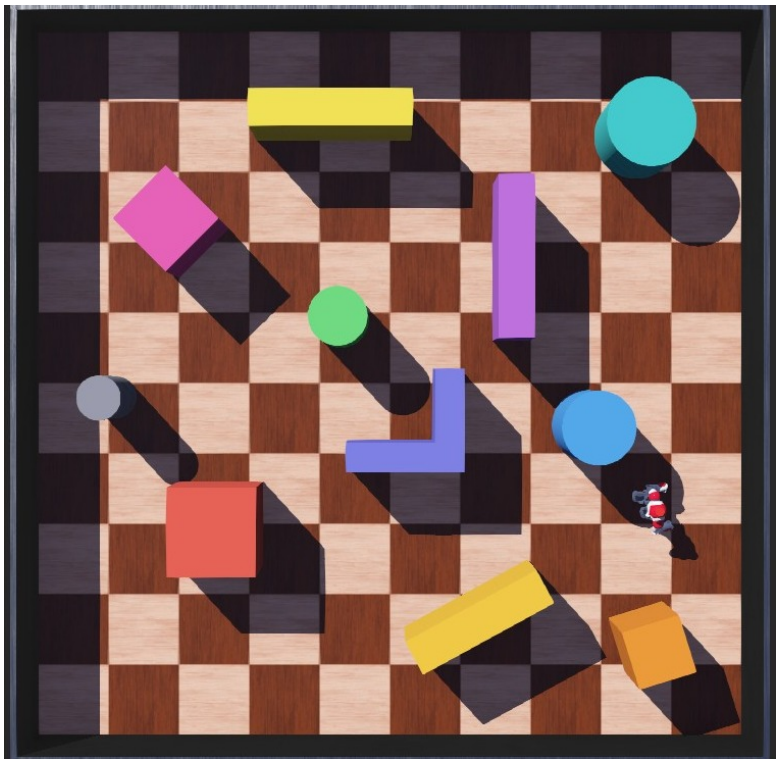
dengan s(·) fungsi smoothstep di atas. Rumus belok kanan sama, dengan batas frame 20, 33, dan 46. Pulsa bernilai nol di batas siklus, jadi siklus lurus, belok kiri, dan belok kanan dapat disambung bebas. Amplitudo 0.4–0.5 rad sudah diuji, tetapi hasilnya tidak konsisten karena kaki tergelincir. Karena itu dipilih 0.25 rad. Putar di tempat 20° dan 10° dibuat dengan cara serupa: hanya sendi HipYawPitch dari TurnLeft40/TurnRight40 yang diskalakan 0.5 dan 0.25.

Tabel 3. Primitif gerak hasil gait engine (diukur di simulasi, tanpa jatuh)

Gerakan	Durasi	Hasil terukur (rata-rata)
jalan_lurus (1 siklus)	2.04 s	+0.18 m, 0°
jalan_kiri / jalan_kanan (1 siklus)	2.04 s	+0.16 m dan +17° / +0.17 m dan −13°
jalan_mulai / jalan_berhenti	2.04 / 2.68 s	+0.10 m / +0.18 m
putar_kiri10 / putar_kanan10	2.9 s	+10.9° / −11.3°
putar_kiri20 / putar_kanan20	2.9 s	+20.7° / −20.7°
putar_kiri40 / putar_kanan40	2.9 s	+39.4° / −39.7°
putar_kiri60 / putar_kanan60	4.5 s	+59.1° / −59.1°
putar_balik	9.0 s	+196°
mundur / maju_kecil	2.6 s	−0.10 m / +0.10 m

5. Perancangan Arena

Arena berukuran 5 × 5 m dengan 12 obstacle setinggi 0.8 m. Obstacle harus lebih tinggi dari Lidar (0.57 m) agar terlihat. Jarak antar obstacle bervariasi: sebagian celah bisa dilewati humanoid, sebagian sengaja terlalu sempit, termasuk sudut cekung siku-L dan kantong sempit di dekat tiang.



Gambar 4. Tampak atas arena 5 × 5 m dengan 12 obstacle.

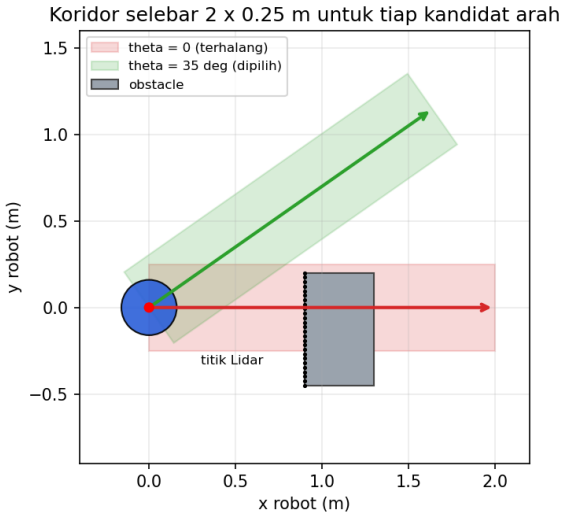
6. Algoritma Navigasi

6.1 Pemilihan Arah dengan Koridor

Keputusan diambil setiap kali satu gerakan selesai. Saat berjalan, keputusan diambil di setiap batas siklus (setiap 2.04 s). Untuk 36 kandidat arah θ (−180° sampai 170°, kelipatan 10°) dihitung jarak bebas koridor selebar 2w (w = 0.25 m, yaitu setengah lebar bahu dan lengan humanoid ditambah margin). Setiap titik Lidar (x, y) diproyeksikan:

$$\ell = x \cos\theta + y \sin\theta \text{ (searah } \theta\text{)}, \quad d = -x \sin\theta + y \cos\theta \text{ (menyamping)}$$

$$\text{bebas}(\theta) = \min \{ \ell : \ell > 0 \text{ dan } |d| < w \}, \text{ dibatasi maksimum 3.5 m}$$



Gambar 5. Koridor kandidat arah. Arah lurus terhalang obstacle, arah 35° bebas.

6.2 Skor Arah dan Eksplorasi Kebaruan

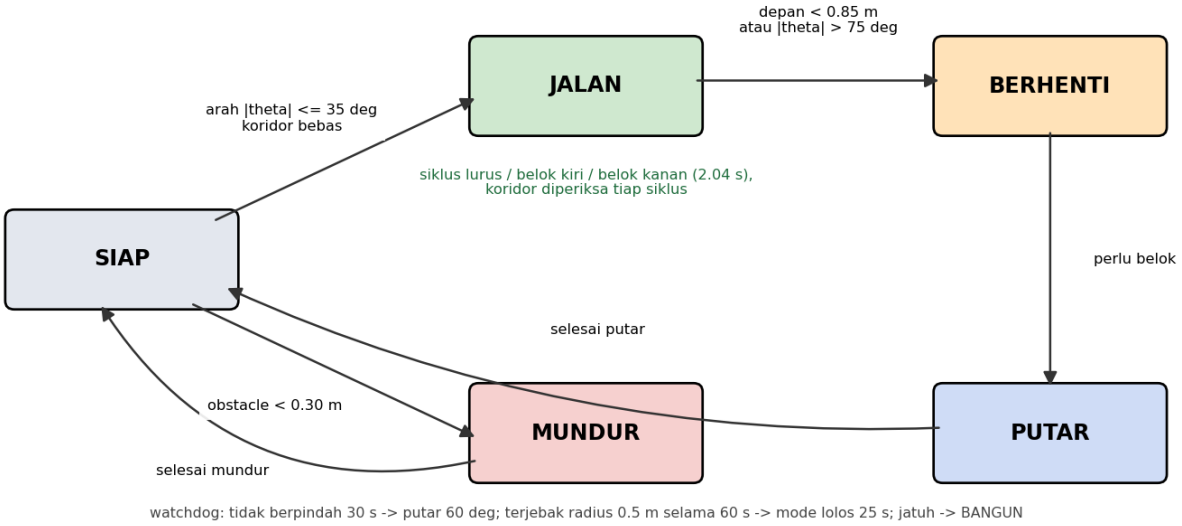
Arah dengan bebas(θ) < 0.85 m tidak dipertimbangkan (0.6 m jika robot sudah berputar 360° tanpa menemukan jalan). Untuk arah lainnya dihitung skor berikut, lalu dipilih arah θ* dengan skor terbesar:

$$\text{skor}(\theta) = \min(\text{bebas}, 2)/2 + 0.6 \cdot K(\theta) - 0.35 \cdot |\theta|/\pi - 0.25 \cdot |\text{wrap}(\psi + \theta - \psi_{\text{tujuan}})|/\pi$$

- **min(bebas, 2)/2**: mengutamakan arah yang lapang. Nilainya 0 sampai 1 dan jarak di atas 2 m dianggap sama.
- **K(θ), kebaruan**: rata-rata $1/(1 + n_c)$ untuk sel grid 0.25 m yang dilalui garis arah θ, mulai 0.5 m sampai min(bebas, 2.5 m). n_c adalah jumlah kunjungan sel c (dicatat setiap 0.5 s dari posisi GPS). Sel yang belum pernah dikunjungi bernilai 1.
- **|θ|/π**: penalti belok, karena humanoid lambat berbelok.
- **Persistensi**: ψ adalah arah hadap robot (yaw IMU) dan ψ_{tujuan} adalah arah tujuan pada keputusan sebelumnya (frame dunia). Suku ini mencegah tujuan berganti-ganti sehingga robot tidak berputar bolak-balik. Fungsi wrap(·) memetakan sudut ke rentang (−π, π].

Sebagai pembanding, **mode acak** mengganti suku kebaruan dengan $-0.3 \cdot |\text{wrap}(\psi + \theta - \psi_{\text{acak}})|/\pi$, dengan ψ_{acak} arah acak yang diganti setiap 20 s. Semua aturan keselamatan sama untuk kedua mode.

6.3 State Machine



Gambar 6. State machine humanoid.

6.4 Aturan Keselamatan

Tabel 4. Aturan keselamatan dan ambangnya

Aturan	Ambang	Alasan
Lanjut satu siklus jalan	koridor ≥ 0.85 m searah langkah, koridor lurus ≥ 0.70 m saat berbelok	Satu siklus (0.18 m) + fase berhenti (0.18 m) + badan + margin
Mulai jalan	koridor depan ≥ 0.60 m, atau ≥ 0.45 m jika samping (30°–120°) ≥ 0.35 m	Fase mulai 0.10 m + berhenti 0.18 m. Lengan bisa menyenggol obstacle di samping
Berputar di tempat	semua titik ≥ 0.33 m. Jika tidak: mundur atau maju kecil dulu	Lengan dan ayunan badan saat berputar
Mundur	obstacle di depan (±50°) < 0.30 m dan belakang ≥ 0.45 m	Terlalu dekat untuk berbelok
Sonar	< 0.30 m, berlaku hanya jika Lidar depan < 0.6 m	Pancaran sonar lebar sehingga sering menangkap obstacle di samping
Belokan besar (|θ| > 90°)	putar ke sisi yang lebih lega, arah dikunci	Mencegah putar kiri-kanan bolak-balik
Watchdog macet	pindah < 0.15 m dalam 30 s → putar 60°	Keluar dari kebuntuan
Watchdog wilayah	selalu dalam radius 0.5 m selama 60 s → mode lolos 25 s	Keluar dari kantong sempit dengan memilih arah paling lega
Jatuh	|roll| atau |pitch| > 45° → gerakan bangun	Pemulihan

7. Pengujian dan Hasil

7.1 Metode

World dilengkapi robot **evaluator** (Supervisor) yang hanya mengamati. **Tabrakan** dideteksi dari titik kontak fisik robot yang lebih tinggi dari 3 cm, karena telapak kaki menyentuh lantai di z ≈ 0. Metode ini sudah divalidasi dengan robot uji yang sengaja ditabrakkan. **Jatuh** dideteksi saat ketinggian torso turun di bawah 0.22 m (tinggi saat berdiri sekitar 0.30 m). **Clearance** adalah jarak analitik dari pusat robot ke permukaan obstacle terdekat, dikurangi radius pembungkus 0.18 m. **Coverage** adalah persentase sel grid 20 cm yang dapat dilalui

dan pernah dikunjungi.

Matriks uji: 16 posisi awal (2 set × 8) × 2 mode (kebaruan dan acak) × 10 menit = 32 uji, total 5.3 jam simulasi, dijalankan otomatis dengan `tools/stress_test.sh`.

7.2 Hasil

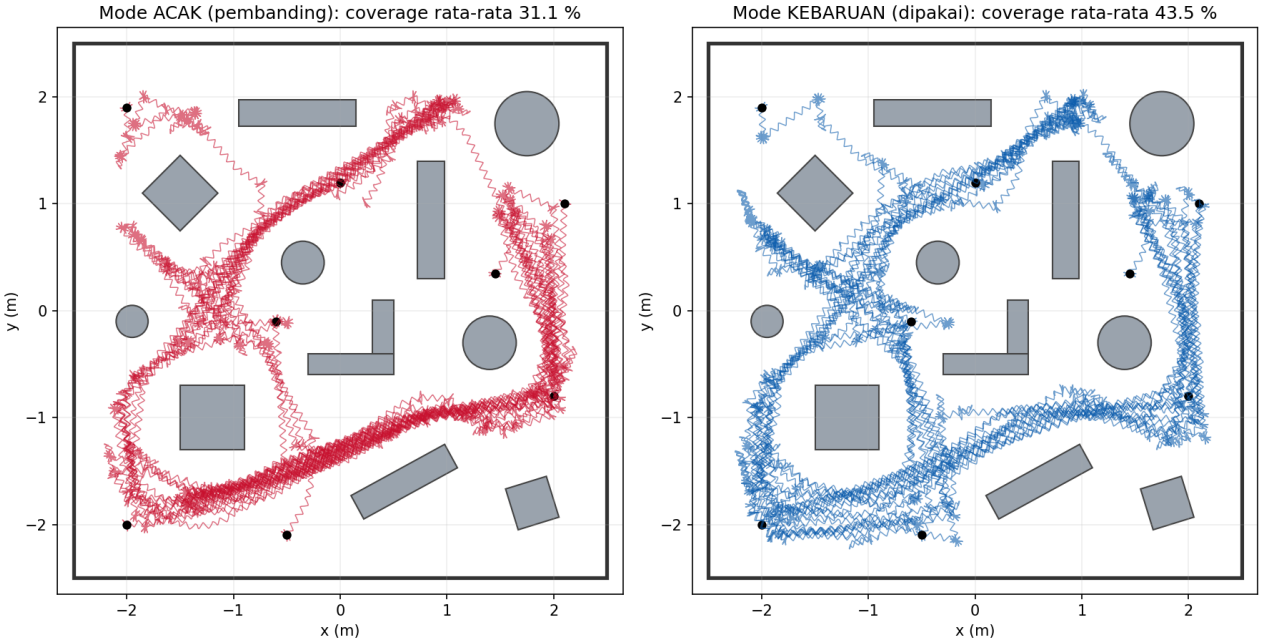
Tabel 5. Ringkasan hasil uji

Mode	Uji	Tabrakan	Jatuh	Jarak total	Clearance min*	Coverage rata-rata (min–maks)
Kebaruan	16	**0**	**0**	681 m	1.4 cm	**42.9 %** (27.9–48.8)
Acak	16	0	0	704 m	2.5 cm	30.3 % (19.4–37.3)

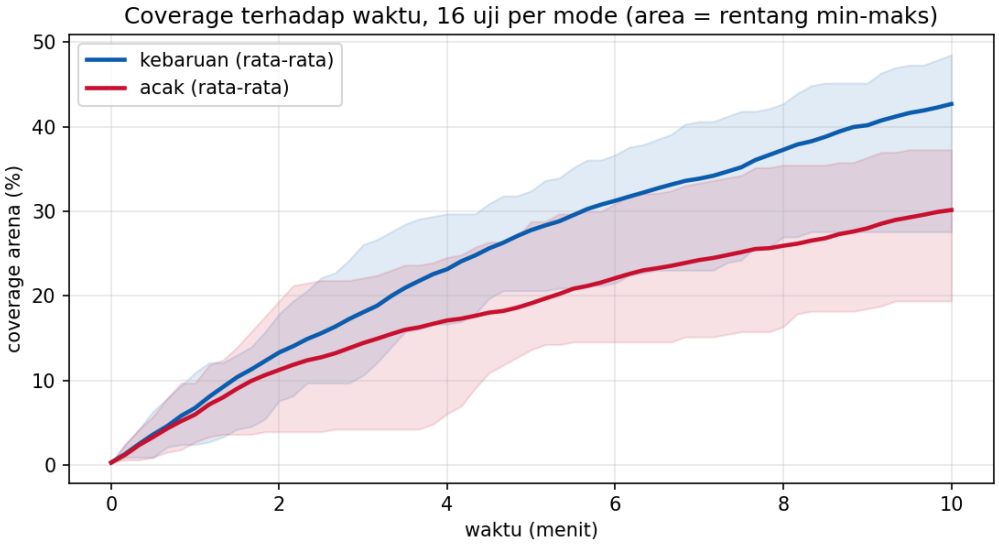
Jarak dihitung dari posisi setiap 1 detik, sehingga ayunan badan ke kiri dan kanan tidak ikut terhitung.

*Terhadap lingkaran pembungkus 0.18 m. Nilai mendekati 0 atau sedikit negatif tidak berarti tabrakan, karena badan humanoid lebih ramping dari lingkaran tersebut. Tabrakan diukur terpisah dari kontak fisik.

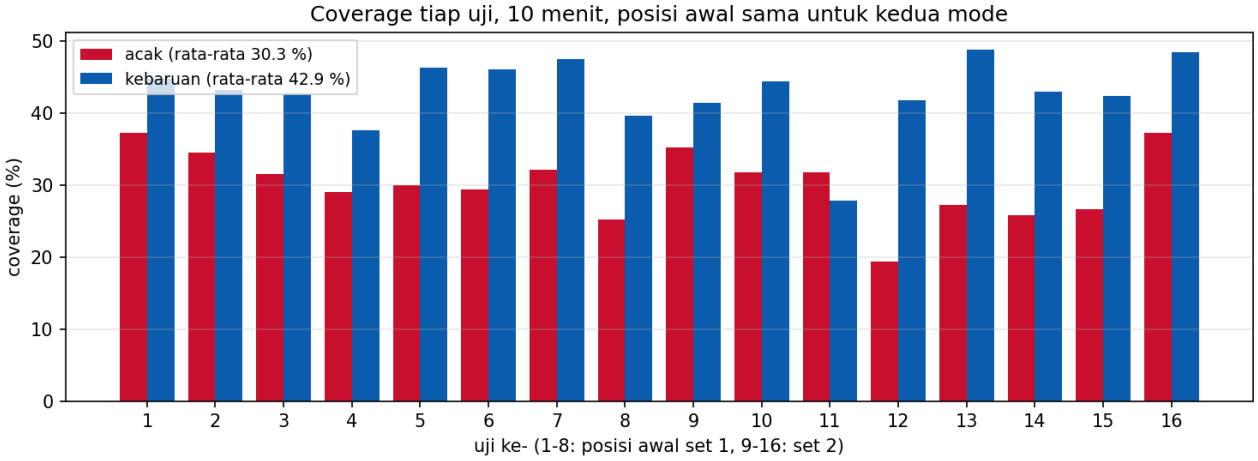
Selama 5.3 jam simulasi dengan total jarak tempuh 1384 m, humanoid **tidak pernah menabrak (0) dan tidak pernah jatuh (0)**. Eksplorasi kebaruan menghasilkan coverage rata-rata 42.9 %, sedangkan mode acak 30.3 %. Artinya coverage **naik 42 %** dengan posisi awal, durasi, dan aturan keselamatan yang sama.



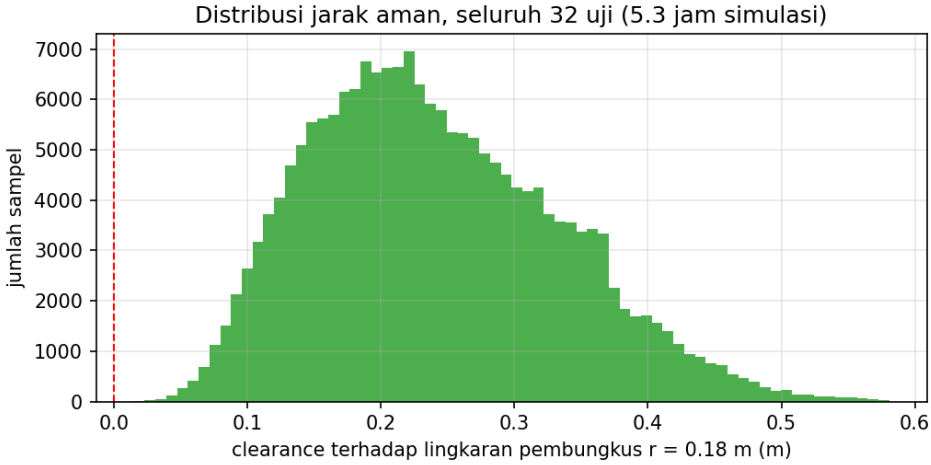
Gambar 7. Lintasan 8 uji (set 1). Kiri: mode acak. Kanan: mode kebaruan, yang menjangkau lebih banyak area dan celah di antara obstacle.



Gambar 8. Coverage terhadap waktu, rata-rata 16 uji per mode.



Gambar 9. Coverage setiap uji dengan posisi awal yang sama untuk kedua mode.



Gambar 10. Distribusi jarak aman seluruh uji.

Tabel 6. Hasil per uji (10 menit)

#	Posisi awal	Kebaruan: jarak	tabrakan/jatuh	coverage	Acak: coverage
1	(−2.0, −2.0, 45°)	42.2 m	0 / 0	44.8 %	37.3 %
2	(2.1, 1.0, −132°)	43.3 m	0 / 0	43.3 %	34.5 %
3	(0.0, 1.2, 0°)	43.8 m	0 / 0	42.7 %	31.5 %
4	(2.0, −0.8, 180°)	41.1 m	0 / 0	37.6 %	29.1 %
5	(−2.0, 1.9, −29°)	43.0 m	0 / 0	46.4 %	30.0 %
6	(−0.6, −0.1, 90°)	43.0 m	0 / 0	46.1 %	29.4 %
7	(−0.5, −2.1, 0°)	41.9 m	0 / 0	47.6 %	32.1 %
8	(1.45, 0.35, 115°)	41.9 m	0 / 0	39.7 %	25.2 %
9	(−2.1, 0.6, 0°)	42.9 m	0 / 0	41.5 %	35.2 %
10	(0.0, −1.1, 90°)	42.2 m	0 / 0	44.5 %	31.8 %
11	(1.1, −0.9, 143°)	39.2 m	0 / 0	27.9 %	31.8 %
12	(−1.0, 0.0, −57°)	43.4 m	0 / 0	41.8 %	19.4 %
13	(0.3, 0.9, 57°)	43.7 m	0 / 0	48.8 %	27.3 %
14	(2.15, −1.0, 90°)	42.3 m	0 / 0	43.0 %	25.8 %
15	(−0.9, −2.1, 57°)	42.9 m	0 / 0	42.4 %	26.7 %
16	(0.5, 2.1, 0°)	43.8 m	0 / 0	48.5 %	37.3 %

8. Masalah yang Ditemukan dan Solusinya

Pengembangan dilakukan secara iteratif. Setiap perubahan diuji ulang dengan uji ketahanan 8 posisi awal. Tabel berikut merangkum masalah nyata yang ditemukan dari data uji beserta perbaikannya.

Tabel 7. Masalah, penyebab, dan solusi

Masalah (dari data)	Penyebab	Solusi
Robot jatuh di detik 1.2 (Forwards50)	Motion melompat ke pose lutut 1.22 rad	Transisi smoothstep 0.4 s
Belok hanya bisa sambil diam (lambat)	Tidak ada motion belok sambil jalan	Pulsa HipYawPitch per siklus
Belok tidak konsisten pada amplitudo besar	Kaki tergelincir	Amplitudo dibatasi 0.25 rad
Lengan menyenggol dinding saat berputar	Clearance putar terlalu kecil	Clearance 0.33 m dan putar ke sisi lega
Berputar kiri-kanan di sudut ruang (9 menit)	Target di belakang berpindah sisi	Kunci arah putar
Overshoot target kecil (±20°)	Putar minimum 40°	Putar 20° (HipYawPitch × 0.5)
Putar ±20° bolak-balik di dekat celah	Resolusi kandidat arah 10° lebih halus dari putar 20°	Putar 10° (HipYawPitch × 0.25)
Senggolan saat mulai jalan lalu berhenti	Syarat mulai jalan terlalu longgar	Mulai jalan perlu 0.60 m, atau 0.45 m jika samping lega
Maju-mundur di dekat tiang	Sonar menangkap obstacle di samping	Sonar perlu konfirmasi Lidar
Terjebak di kantong sempit (63–71 % waktu)	Skor tidak pernah memilih jalan keluar	Watchdog wilayah dan mode lolos
43 % waktu habis untuk putar di tempat	Belok hanya saat berhenti	Belok sambil jalan sampai ±75°

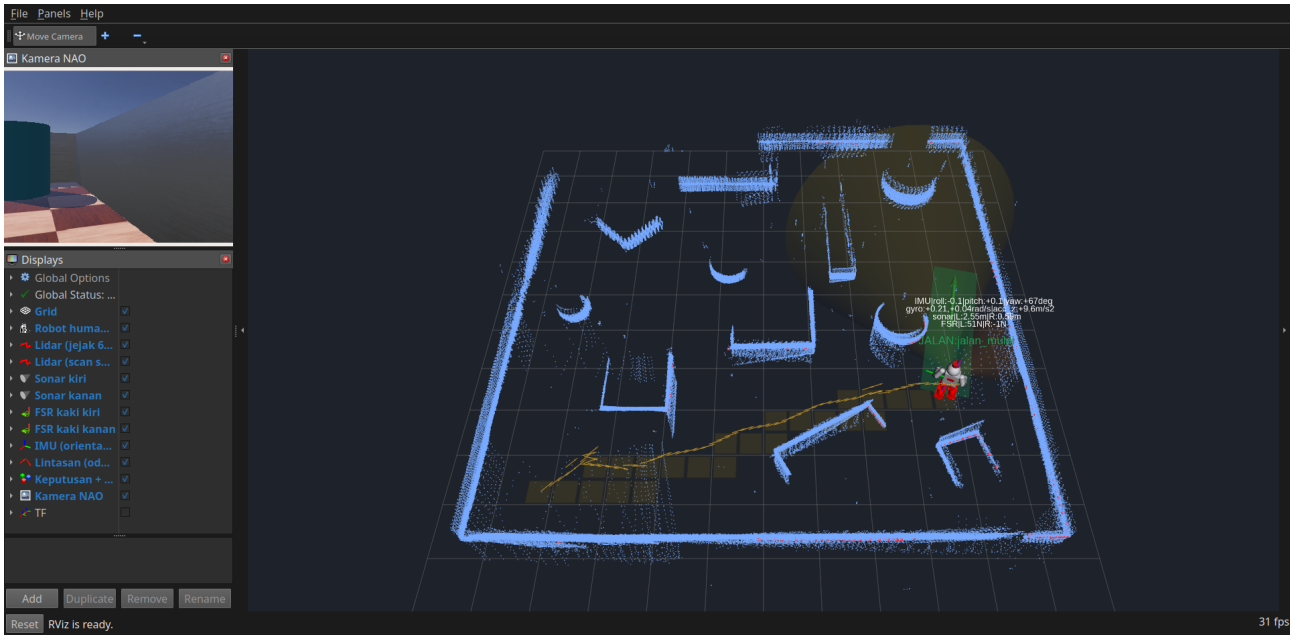
9. Visualisasi Semua Sensor di RViz2 (ROS 2)

Controller memiliki jembatan ROS 2 opsional (`ros_bridge.py`) yang aktif otomatis jika Webots dijalankan dari terminal yang sudah men-*source* ROS 2 Jazzy. Model robot ditampilkan dari URDF. Webots dapat mengekspor URDF, tetapi sendi bahu dan leher (Hinge2Joint) belum didukung, sehingga lengan, kepala, Lidar, dan kamera ditambahkan oleh `tools/make_urdf.py` memakai dimensi kinematika NAO. Hasilnya, model humanoid di RViz2 bergerak sesuai sudut sendi asli robot.

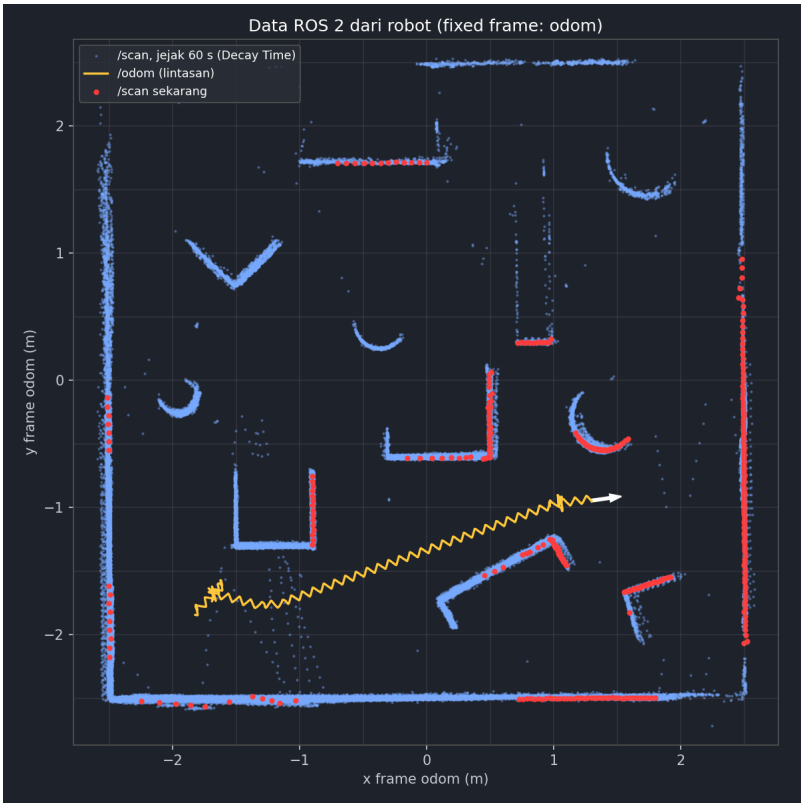
Tabel 8. Topic ROS 2 yang dipublish

Topic	Tipe	Isi	Tampilan RViz2
/scan	LaserScan	Lidar 360°	titik merah + jejak 60 s
/sonar/left, /sonar/right	Range	Sonar dada	kerucut kuning
/imu	Imu	Orientasi + gyro + accelerometer	sumbu imu_body + panel teks
/fsr/left, /fsr/right	WrenchStamped	Gaya telapak kaki	panah gaya
/camera/image_raw	Image	Kamera atas NAO	panel gambar
/joint_states	JointState	24 sudut sendi	model robot bergerak
/odom, /tf	Odometry, tf2	GPS + IMU, frame semua link	lintasan
/oa/decision	MarkerArray	Koridor, arah, state, sel dikunjungi	marker berwarna

Hasil verifikasi: semua topic sensor terkirim sekitar 24.5 Hz (kamera sekitar 5 Hz). Accelerometer membaca +9.93 m/s² pada sumbu z saat berdiri (sudah disesuaikan dengan orientasi pemasangan sensor yang terbalik). FSR kaki membaca sekitar 50 N saat satu kaki menumpu. Frame Lidar berada 0.57 m di atas lantai, sesuai rancangan.



Gambar 11. Tangkapan layar RViz2 saat simulasi berjalan: model humanoid (URDF + joint_states), jejak Lidar, kerucut sonar, panah gaya kaki, lintasan, sel yang dikunjungi, panel nilai sensor, dan gambar kamera NAO (kiri atas).



Gambar 12. Data /scan dan /odom yang direkam dari robot. Jejak Lidar membentuk peta arena.

10. Cara Menjalankan

- Instal Webots R2025a dan Python 3. Untuk RViz2, instal juga ROS 2 Jazzy.

- `git clone https://github.com/ahnafyura/webots-obstacle-avoidance-b401.git`
- Webots: `File → Open World`, pilih `worlds/obstacle_avoidance.wbt`, lalu tekan Play.
- RViz2 dengan semua sensor: `tools/rviz.sh`.
- Uji otomatis: `tools/stress_test.sh 600` kebaruan dan `tools/stress_test.sh 600` acak (gunakan `STARTS_SET=2` untuk set posisi awal kedua), lalu `python3 tools/plot_results.py`.

11. Kesimpulan

- Robot humanoid NAO 25 DOF berhasil berjalan stabil memakai gait engine buatan sendiri. Gait engine ini mengendalikan setiap sendi dengan transisi smoothstep, siklus jalan kontinu, dan belok sambil berjalan. Dalam 5.3 jam uji, robot tidak pernah jatuh.
- Lidar 360° di atas kepala dengan pemilihan arah berbasis koridor selebar badan menghasilkan **0 tabrakan** dari 32 uji, dengan total jarak tempuh 1384 m.
- Eksplorasi kebaruan menaikkan coverage dari 30.3 % menjadi 42.9 % (+42 %).
- Semua sensor (Lidar, sonar, IMU, gyro, accelerometer, FSR, kamera, posisi sendi) dan model robot dapat dipantau langsung di RViz2.
- Pengembangan selanjutnya: lokalisasi tanpa GPS (SLAM), gait dengan umpan balik keseimbangan (ZMP), dan obstacle bergerak.

Lampiran A. Controller humanoid_avoidance.py

```
"""
Controller : humanoid_avoidance
Tugas      : Pelatihan Lab B401 - Robot Obstacle Avoidance (Webots, Python)

Robot      : humanoid NAO (25 DOF). Setiap sendi dikendalikan langsung oleh gait.py.
Sensor     : Lidar 360 derajat di atas kepala (sensor utama)
             2 sonar dada (Sonar/Left, Sonar/Right) sebagai lapisan safety
             InertialUnit (orientasi), GPS (posisi untuk memori eksplorasi)
             Gyro, Accelerometer, sensor gaya kaki (FSR), kamera, sensor posisi sendi
             (semua dikirim ke RViz2 lewat ros_bridge.py)

Algoritma  : keputusan diambil di batas setiap gerakan (setiap siklus jalan 2.04 s).
  1. Scan Lidar diubah ke titik (x, y) frame robot. Untuk 36 kandidat arah theta
     dihitung JARAK BEBAS koridor selebar badan humanoid + margin (2 x 0.22 m).
  2. Skor tiap arah:
        skor(theta) = min(bebas, 2 m)/2 m + W_BARU * kebaruan(theta) - 0.35 * |theta|/pi
     kebaruan(theta) = rata-rata 1/(1 + kunjungan) sel grid 0.25 m yang dilewati
     garis arah theta (sampai jarak bebas). Robot mengutamakan area yang belum
     dikunjungi -> coverage naik (dibandingkan mode acak, lihat laporan).
  3. State machine:
        SIAP   : berdiri, memilih gerakan berikutnya
        JALAN  : siklus jalan lurus / belok kiri / belok kanan, diperpanjang selama
                 koridor depan bebas >= 0.85 m dan arah terbaik |theta| <= 40 derajat
        BERHENTI, PUTAR (40/60 derajat di tempat), MUNDUR, BANGUN (jika jatuh)
"""

import math
import os
import random

from controller import Robot

from gait import Gait

# ---------------------------------------------------------------------------
# Parameter
# ---------------------------------------------------------------------------
HALF_W = 0.25              # m, setengah lebar koridor (bahu+lengan NAO ~0.16 m + margin)
LOOKAHEAD = 2.0            # m, jarak bebas maksimum yang dihargai dalam skor
FREE_CAP = 3.5             # m, batas perhitungan jarak bebas koridor
CONTINUE_FREE = 0.85       # m, koridor minimum (searah langkah) untuk satu siklus jalan lagi
STEER_FRONT = 0.70         # m, koridor lurus minimum saat siklus berbelok menjauhi obstacle
START_FREE = 0.85          # m, koridor minimum untuk mulai jalan ke suatu arah
BACKUP_DIST = 0.30         # m, obstacle sedekat ini di depan -> mundur
ROTATE_SAFE = 0.33         # m, clearance minimal untuk berputar di tempat (lengan + ayunan badan)
STEER_DEADBAND = math.radians(7)
WALK_MAX_ANGLE = math.radians(75)   # belok sambil jalan sampai target sejauh ini
K_TURN = float(os.environ.get('OA_K_TURN', '0.35'))       # penalti belok (per pi radian)
W_NOVELTY = float(os.environ.get('OA_W_NOV', '0.6'))      # bobot kebaruan (mode 'kebaruan')
NOVELTY_RANGE = float(os.environ.get('OA_NOV_RANGE', '2.5'))  # m, jarak sampel kebaruan
K_KEEP = float(os.environ.get('OA_K_KEEP', '0.25'))       # penalti ganti arah tujuan (frame dunia)
CELL = 0.25                # m, ukuran sel memori kunjungan
CANDIDATES = [math.radians(a) for a in range(-180, 180, 10)]
SONAR_STOP = 0.30          # m, sonar membaca lebih dekat -> berhenti/mundur
SONAR_START = float(os.environ.get("OA_SONAR_START", "0.0"))  # m, sonar minimal untuk mulai jalan (0 = tidak dipakai)
START_FRONT = 0.60         # m, koridor depan minimal untuk mulai jalan
START_FRONT_SIDE_CLEAR = 0.45  # m, boleh lebih pendek jika samping lega
SIDE_CLEAR = 0.35          # m, samping dianggap lega
                           #    (mulai 0.10 m + berhenti 0.18 m + badan + margin)
REGION_WINDOW = 60.0       # s, watchdog wilayah: jendela waktu
REGION_RADIUS = 0.5        # m, robot dianggap terjebak jika selalu dalam radius ini
ESCAPE_TIME = 25.0         # s, lama mode lolos
ESCAPE_FREE = 1.2          # m, koridor minimum yang dicari saat mode lolos
LOG_PERIOD = 1.0

# Urutan sinar Lidar 360 di Webots: indeks 0 = belakang (+180 deg), lalu searah jarum
# jam (90 = kiri, 180 = depan, 270 = kanan). Sudut sinar ke-i: pi - i*(2*pi/N).


def clamp(x, lo, hi):
    return lo if x < lo else hi if x > hi else x


def wrap(a):
    return math.atan2(math.sin(a), math.cos(a))

class Humanoid:
```

```
def __init__(self):
    self.robot = Robot()
    self.ts = int(self.robot.getBasicTimeStep())
    self.dt = self.ts / 1000.0
    here = os.path.dirname(os.path.abspath(__file__))
    self.gait = Gait(self.robot, self.dt, os.path.join(here, 'motions'))

    dev = self.robot.getDevice
    self.lidar = dev('lidar')
    self.lidar.enable(self.ts)
    self.lidar.enablePointCloud()          # untuk 'Show Lidar Point Cloud' di Webots
    self.n_rays = self.lidar.getHorizontalResolution()
    self.fov = self.lidar.getFov()
    self.max_range = self.lidar.getMaxRange()
    step = self.fov / self.n_rays
    self.ray_angles = [self.fov / 2.0 - i * step for i in range(self.n_rays)]
    self.ray_cos = [math.cos(a) for a in self.ray_angles]
    self.ray_sin = [math.sin(a) for a in self.ray_angles]

    self.sonar_l, self.sonar_r = dev('Sonar/Left'), dev('Sonar/Right')
    self.imu = dev('imu_body')
    self.gps = dev('gps')
    self.gyro = dev('gyro')
    self.acc = dev('accelerometer')
    self.fsr_l, self.fsr_r = dev('LFsr'), dev('RFsr')
    for d in (self.sonar_l, self.sonar_r, self.imu, self.gps, self.gyro, self.acc,
              self.fsr_l, self.fsr_r):
        d.enable(self.ts)

    self.mode = os.environ.get('OA_EXPLORE', 'kebaruan')   # 'kebaruan' atau 'acak'
    random.seed(int(os.environ.get('OA_SEED', '401')))
    self.visits = {}
    self.visit_timer = 0.0
    self.wander = 0.0
    self.wander_timer = 0.0
    self.state = 'SIAP'
    self.walking = False
    self.time = 0.0
    self.log_timer = 0.0
    self.decision = (0.0, 0.0)       # (theta, jarak bebas) terakhir
    self.stats = {'mundur': 0, 'putar': 0, 'jatuh': 0, 'lolos': 0}
    self.progress = []               # (waktu, x, y) untuk watchdog macet
    self.goal_heading = 0.0          # arah tujuan terakhir (frame dunia), untuk persistensi
    self.turn_lock = None            # arah putar dikunci sampai robot mulai jalan
    self.turn_count = 0              # jumlah putar beruntun tanpa berjalan
    self.region = []                 # (waktu, x, y) 60 s terakhir untuk watchdog wilayah
    self.escape_timer = 0.0          # > 0: mode lolos dari area sempit

    self.ros = None
    if os.environ.get('OA_ROS', '1') != '0':
        try:
            from ros_bridge import RosBridge
            self.ros = RosBridge(self)
        except ImportError:
            pass

# ------------------------------------------------------------------ sensor
def scan(self):
    out = []
    for r in self.lidar.getRangeImage():
        if r != r or r == float('inf') or r <= 0.0 or r > self.max_range:
            r = self.max_range
        out.append(r)
    return out

def pose(self):
    x, y, z = self.gps.getValues()
    roll, pitch, yaw = self.imu.getRollPitchYaw()
    return x, y, z, roll, pitch, yaw

def corridor_free(self, scan, theta, half_width=HALF_W):
    ct, st = math.cos(theta), math.sin(theta)
    free = FREE_CAP
    for i, r in enumerate(scan):
        if r >= FREE_CAP + half_width:
            continue
        x, y = r * self.ray_cos[i], r * self.ray_sin[i]
        longi = x * ct + y * st
        if longi <= 0.0:
            continue
```

```
            if abs(-x * st + y * ct) < half_width and longi < free:
                free = longi
        return free

    def sector_min(self, scan, a_min, a_max):
        m = self.max_range
        for i, r in enumerate(scan):
            if a_min <= self.ray_angles[i] <= a_max and r < m:
                m = r
        return m

    # ------------------------------------------------------------ eksplorasi
    def cell(self, x, y):
        return (int(math.floor(x / CELL)), int(math.floor(y / CELL)))

    def novelty(self, x, y, heading, free):
        dist = min(free, NOVELTY_RANGE)
        if dist < 0.5:
            return 0.0
        vals, d = [], 0.5
        while d <= dist:
            c = self.cell(x + d * math.cos(heading), y + d * math.sin(heading))
            vals.append(1.0 / (1.0 + self.visits.get(c, 0)))
            d += CELL
        return sum(vals) / len(vals)

    def choose(self, scan, x, y, yaw, min_free=START_FREE):
        """Pilih arah terbaik. Kembalikan (theta, jarak_bebas, skor)."""
        best = (0.0, 0.0, -1e9)
        for th in CANDIDATES:
            free = self.corridor_free(scan, th)
            if free < min_free:
                continue
            if self.escape_timer > 0.0:
                # mode lolos: cari ruang paling lega, abaikan kebaruan dan penalti belok
                if free < ESCAPE_FREE:
                    continue
                score = free
                if score > best[2]:
                    best = (th, free, score)
                continue
            score = (min(free, LOOKAHEAD) / LOOKAHEAD - K_TURN * abs(th) / math.pi
                     - K_KEEP * abs(wrap(yaw + th - self.goal_heading)) / math.pi)
            if self.mode == 'kebaruan':
                score += W_NOVELTY * self.novelty(x, y, yaw + th, free)
            else:
                score -= 0.3 * abs(wrap(yaw + th - self.wander)) / math.pi
            if score > best[2]:
                best = (th, free, score)
        return best

    # ------------------------------------------------------------ keputusan
    def decide(self, scan):
        x, y, z, roll, pitch, yaw = self.pose()
        front = self.corridor_free(scan, 0.0)
        front_min = self.sector_min(scan, -math.radians(50), math.radians(50))
        rear_min = min(self.sector_min(scan, math.radians(130), math.pi),
                       self.sector_min(scan, -math.pi, -math.radians(130)))
        all_min = min(scan)
        sonar = min(self.sonar_l.getValue(), self.sonar_r.getValue())
        # sudah berputar 360 deg tanpa menemukan jalan: longgarkan syarat koridor
        min_free = START_FREE if self.turn_count < 7 else 0.6
        # sonar NAO berpancaran lebar (~60 deg) sehingga sering menangkap obstacle di samping.
        # Sonar dipakai sebagai konfirmasi: hanya berlaku jika Lidar juga melihat sesuatu dekat.
        sonar_alarm = sonar < SONAR_STOP and front_min < 0.6
        theta, free, score = self.choose(scan, x, y, yaw, min_free)
        if score > -1e9:
            self.goal_heading = wrap(yaw + theta)
        self.decision = (theta, free if score > -1e9 else front)
        g = self.gait

        if self.walking:
            # koridor diperiksa searah belokan yang akan diambil, bukan hanya lurus ke depan
            left_ok = theta > STEER_DEADBAND and front >= STEER_FRONT \
                and self.corridor_free(scan, math.radians(15)) >= CONTINUE_FREE
            right_ok = theta < -STEER_DEADBAND and front >= STEER_FRONT \
                and self.corridor_free(scan, -math.radians(15)) >= CONTINUE_FREE
            if sonar_alarm or score <= -1e9 or abs(theta) > WALK_MAX_ANGLE \
                    or not (left_ok or right_ok or front >= CONTINUE_FREE):
                g.play('jalan_berhenti')
```

```
            self.walking, self.state = False, 'BERHENTI'
        elif left_ok:
            g.play('jalan_kiri')
            self.state = 'JALAN_KIRI'
        elif right_ok:
            g.play('jalan_kanan')
            self.state = 'JALAN_KANAN'
        else:
            g.play('jalan_lurus')
            self.state = 'JALAN'
        return

    # berdiri
    if (front_min < BACKUP_DIST or (sonar_alarm and front_min < 0.5)) and rear_min > 0.45:
        g.play('mundur')
        self.state = 'MUNDUR'
        self.stats['mundur'] += 1
        return
    stuck = self.stuck()
    # jarak terdekat di samping (30-120 deg kiri dan kanan): lengan bisa menyenggol
    side_min = min(self.sector_min(scan, math.radians(30), math.radians(120)),
                   self.sector_min(scan, -math.radians(120), -math.radians(30)))
    # histeresis: arah dalam +-30 deg -> langsung jalan, koreksi sambil berjalan
    if not stuck and score > -1e9 and abs(theta) <= math.radians(35) \
            and self.corridor_free(scan, theta) >= min_free \
            and sonar >= SONAR_START and (front >= START_FRONT or (
                front >= START_FRONT_SIDE_CLEAR and side_min >= SIDE_CLEAR)):
        g.play('jalan_mulai')
        self.walking, self.state = True, 'JALAN'
        self.turn_lock, self.turn_count = None, 0
        return

    # perlu berputar di tempat: pastikan ada ruang untuk lengan dan ayunan badan
    if all_min < ROTATE_SAFE:
        i_min = scan.index(all_min)
        if abs(self.ray_angles[i_min]) < math.pi / 2 and rear_min > 0.45:
            g.play('mundur')                        # obstacle di depan -> mundur
            self.state = 'MUNDUR'
            self.stats['mundur'] += 1
            return
        if abs(self.ray_angles[i_min]) >= math.pi / 2 and front_min > 0.5:
            g.play('maju_kecil')                    # obstacle di belakang -> maju sedikit
            self.state = 'MAJU_KECIL'
            return

    left_min = self.sector_min(scan, 0.15, math.pi - 0.15)
    right_min = self.sector_min(scan, -math.pi + 0.15, -0.15)
    if score > -1e9 and not stuck and abs(theta) <= math.pi / 2:
        # target di depan: arah putar jelas (ke sisi target), kunci dilepas
        side = 'kiri' if theta > 0 else 'kanan'
        big = abs(theta) > math.radians(50)
    elif self.turn_lock is not None:
        # target di belakang / tidak ada jalan: arah putar dikunci agar tidak bolak-balik
        side = self.turn_lock
        big = score <= -1e9 or stuck or abs(theta) > math.radians(50)
        if stuck:
            self.progress.clear()
    elif score <= -1e9 or stuck:
        side, big = ('kiri' if left_min >= right_min else 'kanan'), True
        self.progress.clear()
    else:
        # belokan besar (> 90 deg): putar ke sisi yang lebih lega, bukan sisi terdekat
        if abs(theta) > math.pi / 2:
            side = 'kiri' if left_min >= right_min else 'kanan'
        else:
            side = 'kiri' if theta > 0 else 'kanan'
        big = abs(theta) > math.radians(50)
    if score > -1e9 and abs(theta) > math.radians(140) and self.turn_lock in (None, 'kiri') \
            and all_min >= ROTATE_SAFE:
        g.play('putar_balik')                       # target di belakang: putar balik (~196 deg)
        self.state = 'PUTAR'
        self.stats['putar'] += 1
        self.turn_lock = 'kiri'
        self.turn_count += 3
        return
    if big:
        size = 60
    elif score > -1e9 and not stuck and abs(theta) <= math.radians(15):
        size = 10                                   # koreksi sangat halus (resolusi kandidat 10 deg)
    elif score > -1e9 and not stuck and abs(theta) <= math.radians(30):
```

```
            size = 20                                   # koreksi halus, tidak overshoot
        else:
            size = 40
        g.play(f'putar_{side}{size}')
        self.state = 'PUTAR'
        self.stats['putar'] += 1
        self.turn_lock = side
        self.turn_count += 1

    def stuck(self):
        """Macet: perpindahan < 0.15 m dalam 30 s terakhir."""
        if len(self.progress) < 2 or self.progress[-1][0] - self.progress[0][0] < 30.0:
            return False
        (_, x0, y0), (_, x1, y1) = self.progress[0], self.progress[-1]
        return math.hypot(x1 - x0, y1 - y0) < 0.15

    # ------------------------------------------------------------------ loop
    def step(self):
        self.time += self.dt
        x, y, z, roll, pitch, yaw = self.pose()

        # memori kunjungan (setiap 0.5 s) dan arah jelajah mode acak (setiap 20 s)
        self.visit_timer += self.dt
        if self.visit_timer >= 0.5 and x == x:
            self.visit_timer = 0.0
            c = self.cell(x, y)
            self.visits[c] = self.visits.get(c, 0) + 1
            self.progress.append((self.time, x, y))
            self.region.append((self.time, x, y))
            while self.region and self.time - self.region[0][0] > REGION_WINDOW:
                self.region.pop(0)
            if self.escape_timer <= 0.0 and self.region \
                    and self.time - self.region[0][0] >= REGION_WINDOW - 1.0:
                mx = sum(p[1] for p in self.region) / len(self.region)
                my = sum(p[2] for p in self.region) / len(self.region)
                if max(math.hypot(p[1] - mx, p[2] - my) for p in self.region) < REGION_RADIUS:
                    self.escape_timer = ESCAPE_TIME
                    self.region.clear()
                    self.stats['lolos'] += 1
            while self.progress and self.time - self.progress[0][0] > 31.0:
                self.progress.pop(0)
        self.escape_timer -= self.dt
        self.wander_timer -= self.dt
        if self.wander_timer <= 0.0:
            self.wander_timer = 20.0
            self.wander = random.uniform(-math.pi, math.pi)

        fallen = abs(roll) > math.radians(45) or abs(pitch) > math.radians(45)
        if fallen and self.state != 'BANGUN':
            self.stats['jatuh'] += 1
            self.walking = False
            self.gait.play('bangun', 0.3)
            self.state = 'BANGUN'
        elif not self.gait.busy():
            self.decide(self.scan())
        self.gait.step()

        if self.ros is not None:
            self.ros.publish()

        self.log_timer += self.dt
        if self.log_timer >= LOG_PERIOD:
            self.log_timer = 0.0
            th, fr = self.decision
            print(f"[t={self.time:6.1f}s] state={self.state:11s} gerak={self.gait.name or '-':14s} "
                  f"arah={math.degrees(th):+5.0f}deg bebas={fr:4.2f}m "
                  f"sonar=({self.sonar_l.getValue():.2f},{self.sonar_r.getValue():.2f})m "
                  f"pos=({x:+.2f},{y:+.2f}) sel={len(self.visits)}"
                  f"{' LOLOS' if self.escape_timer > 0 else ''}", flush=True)

    def run(self):
        print(f"Humanoid B401 siap. Lidar {self.n_rays} sinar, jangkauan {self.max_range} m. "
              f"Mode eksplorasi: {self.mode}", flush=True)
        self.robot.step(self.ts)
        self.gait.go_ready(1.2)
        while self.robot.step(self.ts) != -1:
            self.step()


if __name__ == '__main__':
```

```
Humanoid().run()
```

Lampiran B. Gait engine gait.py

```
"""
Gait engine robot humanoid NAO.

Membaca file .motion (keyframe sudut sendi tiap 40 ms) lalu mengendalikan SETIAP sendi
secara langsung lewat RotationalMotor.setPosition(), dengan tiga perbaikan dibanding
memutar Motion bawaan Webots:

1. Transisi halus (blend). Motion bawaan langsung melompat ke pose pertama; dari posisi
   berdiri tegak lompatan ini membuat robot jatuh (terukur: Forwards50 jatuh ke depan
   pada detik 1.2). Di sini sudut sendi diinterpolasi smoothstep dari target terakhir
   ke pose pertama klip.
2. Siklus jalan kontinu. Forwards50 dipotong menjadi mulai (frame 0-51), siklus
   (51-102, 2 langkah, 2.04 s) dan berhenti (102-170). Frame 51 dan 102 identik
   (selisih 0 rad), jadi siklus dapat diulang tanpa sambungan kasar.
3. Belok sambil jalan. Sendi HipYawPitch diberi satu pulsa per siklus, meniru pola pada
   TurnLeft/TurnRight.motion: dibuka saat kaki yang sesuai terangkat, ditutup saat kaki
   menapak. Pulsa bernilai nol di batas siklus sehingga klip bisa disambung bebas.

Hasil ukur (rata-rata, lihat laporan):
  jalan_lurus  +0.18 m / 2.04 s        jalan_kiri  +0.16 m, +17 deg / 2.04 s
  jalan_kanan  +0.17 m, -13 deg        putar_kiri40/60  +39/+59 deg
  putar_kanan40/60  -40/-59 deg        putar_kiri10/20, kanan10/20  +-11 / +-20.7 deg
  putar_balik  +196 deg (9.0 s)
  mundur  -0.10 m                      maju_kecil  +0.10 m
"""

import os

# Postur siap: lutut ditekuk (pusat massa rendah), lengan di samping badan.
READY = {
    'LHipYawPitch': 0.0, 'LHipRoll': 0.0, 'LHipPitch': -0.524, 'LKneePitch': 1.047,
    'LAnklePitch': -0.524, 'LAnkleRoll': 0.0,
    'RHipYawPitch': 0.0, 'RHipRoll': 0.0, 'RHipPitch': -0.524, 'RKneePitch': 1.047,
    'RAnklePitch': -0.524, 'RAnkleRoll': 0.0,
    'LShoulderPitch': 1.745, 'LShoulderRoll': 0.349, 'LElbowYaw': -1.396, 'LElbowRoll': -1.396,
    'RShoulderPitch': 1.745, 'RShoulderRoll': -0.349, 'RElbowYaw': 1.396, 'RElbowRoll': 1.396,
    'HeadYaw': 0.0, 'HeadPitch': 0.0,
}
JOINTS = list(READY)

LOOP_START, LOOP_END = 51, 102     # frame siklus jalan di Forwards50.motion
TURN_AMP = 0.25                    # rad, amplitudo pulsa HipYawPitch (stabil, terukur)


def _smooth(u):
    return u * u * (3.0 - 2.0 * u)


def load_motion(path, start=0, end=None):
    """Baca file .motion -> (nama sendi, [(waktu_s, [sudut...]), ...])."""
    lines = open(path).read().strip().split('\n')
    joints = lines[0].split(',')[2:]
    frames = []
    for line in lines[1:]:
        parts = line.split(',')
        mm, ss, ms = parts[0].split(':')
        frames.append((int(mm) * 60 + int(ss) + int(ms) / 1000.0, [float(v) for v in parts[2:]]))
    frames = frames[start:end]
    t0 = frames[0][0]
    return joints, [(t - t0, v) for t, v in frames]


class Clip:
    def __init__(self, joints, frames):
        self.joints, self.frames = joints, frames
        self.duration = frames[-1][0]

    def at(self, t):
        f = self.frames
        if t <= 0.0:
            return f[0][1]
        if t >= self.duration:
            return f[-1][1]
        lo, hi = 0, len(f) - 1
        while hi - lo > 1:
            mid = (lo + hi) // 2
            if f[mid][0] <= t:
```

```
                lo = mid
            else:
                hi = mid
        (ta, a), (tb, b) = f[lo], f[hi]
        u = (t - ta) / (tb - ta)
        return [x + (y - x) * u for x, y in zip(a, b)]


def turn_pulse(clip, direction, amp=TURN_AMP):
    """Siklus jalan + satu pulsa HipYawPitch. direction +1 = kiri, -1 = kanan.
    Kiri: buka pada frame 7-20 (ayunan badan ke sisi +), tutup 20-33.
    Kanan: buka 20-33, tutup 33-46. Di luar itu nol, termasuk di batas siklus."""
    iyl, iyr = clip.joints.index('LHipYawPitch'), clip.joints.index('RHipYawPitch')
    a, b, c = (7, 20, 33) if direction > 0 else (20, 33, 46)
    frames = []
    for k, (t, v) in enumerate(clip.frames):
        if a <= k < b:
            s = (k - a) / (b - a)
        elif b <= k < c:
            s = 1.0 - (k - b) / (c - b)
        else:
            s = 0.0
        v = list(v)
        v[iyl] -= amp * _smooth(s)
        v[iyr] -= amp * _smooth(s)
        frames.append((t, v))
    return Clip(clip.joints, frames)


def scale_hip_yaw(clip, k):
    """Putar di tempat dengan sudut lebih kecil: hanya sendi HipYawPitch yang diskalakan.
    TurnLeft40/TurnRight40 x 0.25 -> +-11 deg, x 0.5 -> +-20.7 deg (terukur, stabil)."""
    iyl, iyr = clip.joints.index('LHipYawPitch'), clip.joints.index('RHipYawPitch')
    frames = []
    for t, v in clip.frames:
        v = list(v)
        v[iyl] *= k
        v[iyr] *= k
        frames.append((t, v))
    return Clip(clip.joints, frames)


class Gait:
    # klip yang disambung tanpa blend (bagian dari siklus jalan)
    CHAIN = {'jalan_lurus', 'jalan_kiri', 'jalan_kanan', 'jalan_berhenti'}

    def __init__(self, robot, dt, motion_dir):
        self.dt = dt
        self.motors, self.sensors = {}, {}
        for j in JOINTS:
            self.motors[j] = robot.getDevice(j)
            s = robot.getDevice(j + 'S')
            s.enable(int(round(dt * 1000)))
            self.sensors[j] = s
        p = lambda name: os.path.join(motion_dir, name + '.motion')
        fw = p('Forwards50')
        loop = Clip(*load_motion(fw, LOOP_START, LOOP_END + 1))
        self.clips = {
            'jalan_mulai': Clip(*load_motion(fw, 0, LOOP_START + 1)),
            'jalan_lurus': loop,
            'jalan_kiri': turn_pulse(loop, +1),
            'jalan_kanan': turn_pulse(loop, -1),
            'jalan_berhenti': Clip(*load_motion(fw, LOOP_END)),
            'putar_kiri10': scale_hip_yaw(Clip(*load_motion(p('TurnLeft40'))), 0.25),
            'putar_kanan10': scale_hip_yaw(Clip(*load_motion(p('TurnRight40'))), 0.25),
            'putar_kiri20': scale_hip_yaw(Clip(*load_motion(p('TurnLeft40'))), 0.5),
            'putar_kanan20': scale_hip_yaw(Clip(*load_motion(p('TurnRight40'))), 0.5),
            'putar_kiri40': Clip(*load_motion(p('TurnLeft40'))),
            'putar_kiri60': Clip(*load_motion(p('TurnLeft60'))),
            'putar_kanan40': Clip(*load_motion(p('TurnRight40'))),
            'putar_kanan60': Clip(*load_motion(p('TurnRight60'))),
            'putar_balik': Clip(*load_motion(p('TurnLeft180'))),
            'mundur': Clip(*load_motion(p('Backwards'))),
            'maju_kecil': Clip(*load_motion(p('Forwards'))),
            'bangun': Clip(*load_motion(p('StandUpFromFront'))),
        }
        self.target = dict(READY)
        self.name, self.clip, self.t, self.blend, self.blend_from = None, None, 0.0, 0.0, None

    def joint_positions(self):
```

```
    return {j: s.getValue() for j, s in self.sensors.items()}

def play(self, name, blend=None):
    if blend is None:
        blend = 0.0 if name in self.CHAIN else 0.4
    self.name, self.clip, self.t = name, self.clips[name], 0.0
    self.blend, self.blend_from = blend, dict(self.target)

def go_ready(self, seconds=1.2):
    """Transisi dari posisi sendi saat ini (berdiri tegak) ke postur siap."""
    cur = self.joint_positions()
    self.clips['siap'] = Clip(JOINTS, [(0.0, [cur[j] for j in JOINTS]),
                                       (seconds, [READY[j] for j in JOINTS])])
    self.target = cur
    self.play('siap', 0.0)

def busy(self):
    return self.clip is not None

def step(self):
    c = self.clip
    if c is not None:
        if self.t < self.blend:
            u = _smooth(self.t / self.blend)
            for j, v in zip(c.joints, c.frames[0][1]):
                self.target[j] = self.blend_from[j] + (v - self.blend_from[j]) * u
        else:
            for j, v in zip(c.joints, c.at(self.t - self.blend)):
                self.target[j] = v
            if self.t - self.blend >= c.duration:
                self.clip = None
        self.t += self.dt
    for j, m in self.motors.items():
        m.setPosition(self.target[j])
```